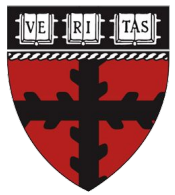
Lecture 18: APIs

AC215/E115

Pavlos Protopapas
Shivas Jayaram



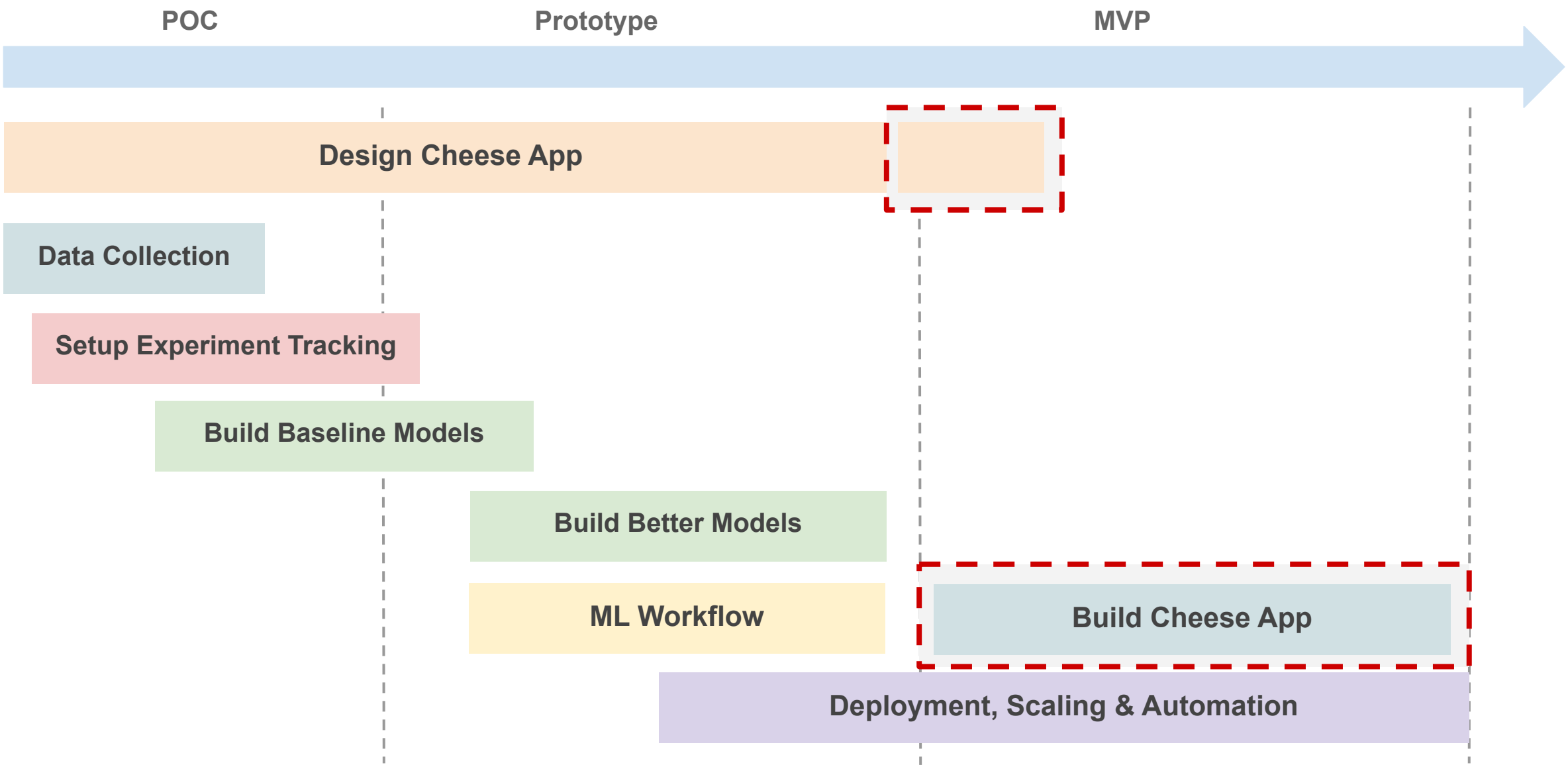
Outline

1. Recap
2. Technical Architecture Review
3. APIs

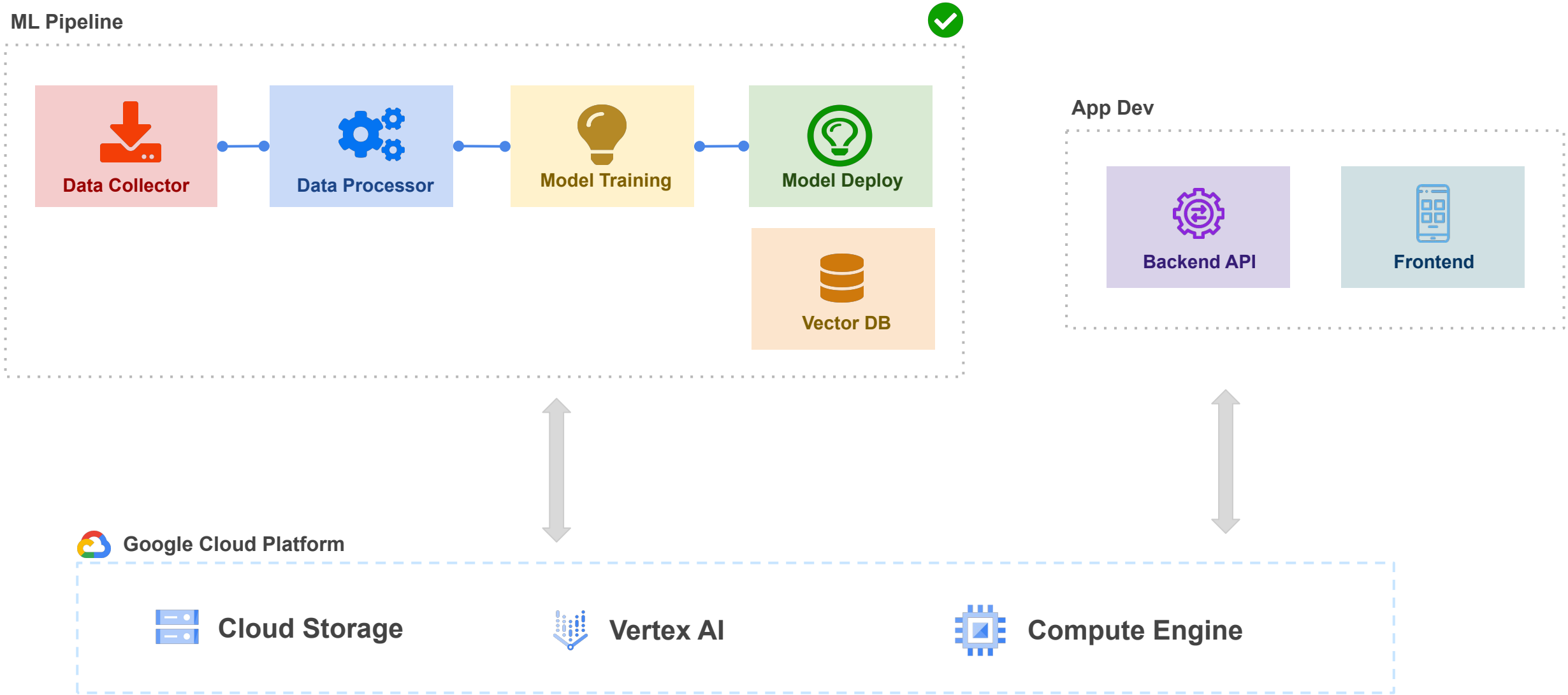
Outline

1. **Recap**
2. Technical Architecture Review
3. APIs

Recap: Cheese App Status



Recap: Cheese App Development



Outline

1. Recap
2. **Technical Architecture Review**
3. APIs

Technical Architecture Review

- Helps design and develop an **AI App**
- High level view from **development** to **deployment**
- Illustrates **interactions** between components/**containers**
- **Blueprint** of the system
  - Helps team members understand the big picture
  - Helps onboarding new team members

Building a Technical Architecture



Building a Technical Architecture



Building a Technical Architecture



Developers:

- Use IDE (VSCode), CLI to build app components
- All development is containerized

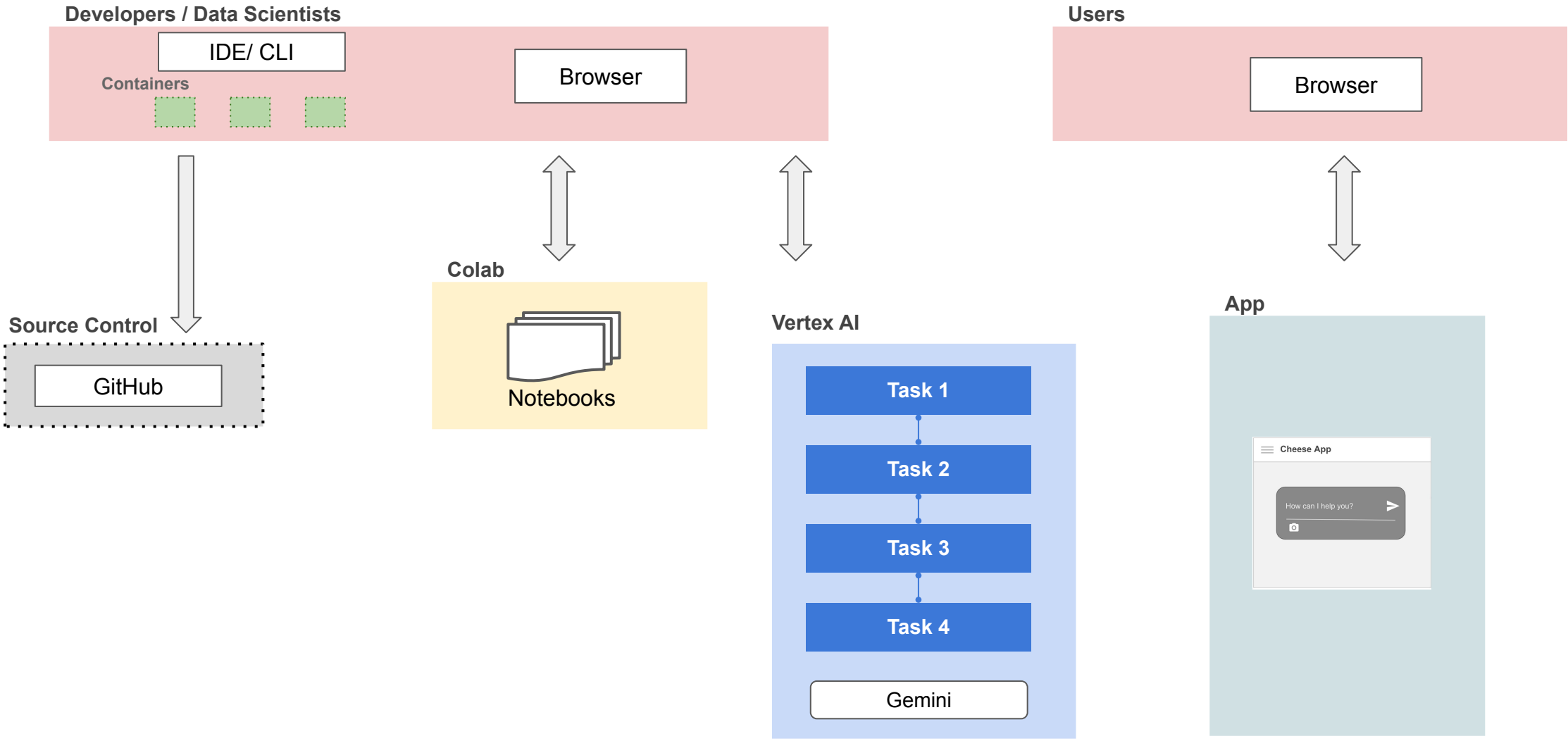
Data Scientists:

- Use Colab/JupyterHub
- EDA on notebooks
- Data & Model experimentation on notebooks
- Use IDE (VSCode), CLI to build ML Tasks
- All development is containerized
- Access LLMs

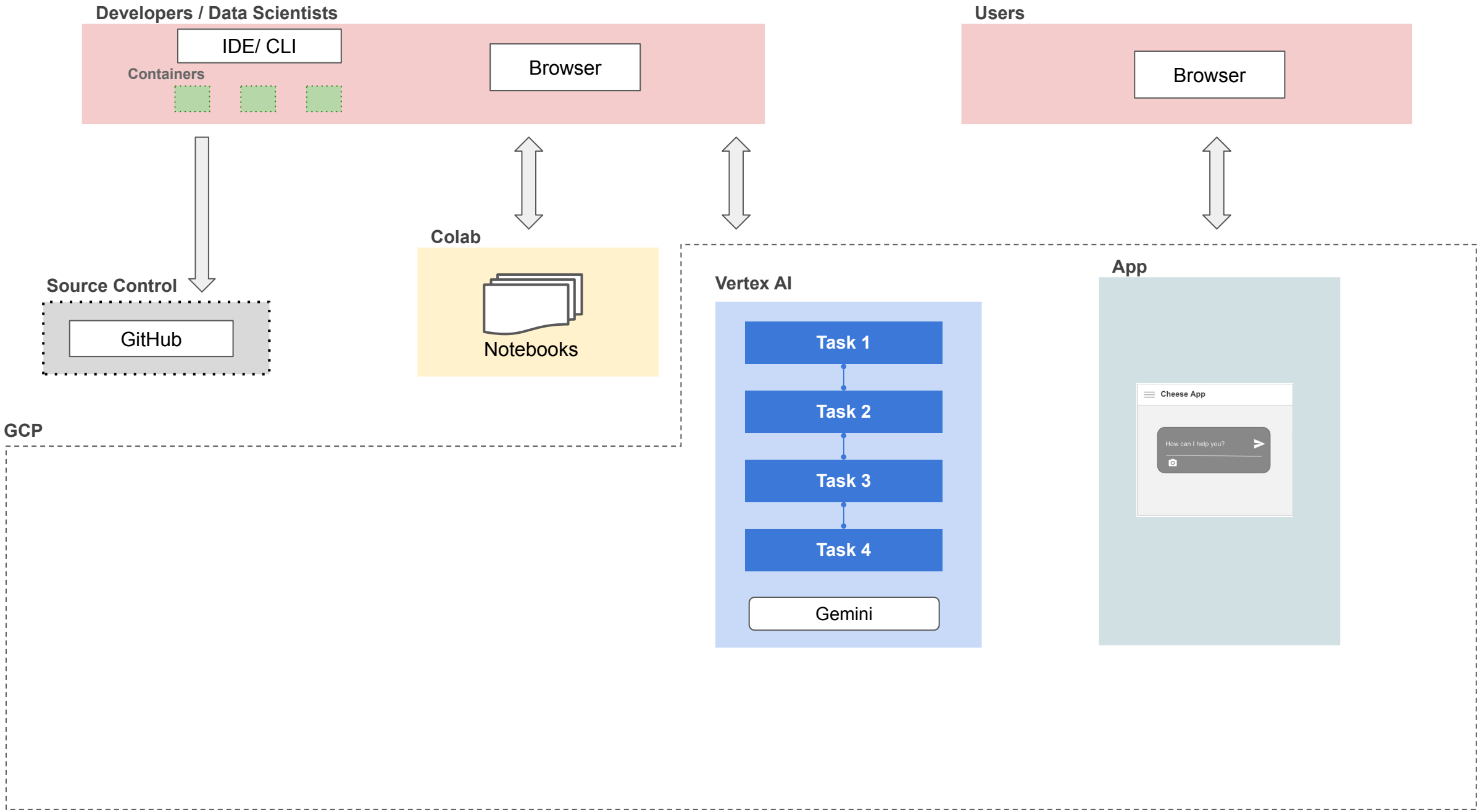
Users:

- Access the App using a browser
- Upload images and view prediction results
- Have a chat conversation

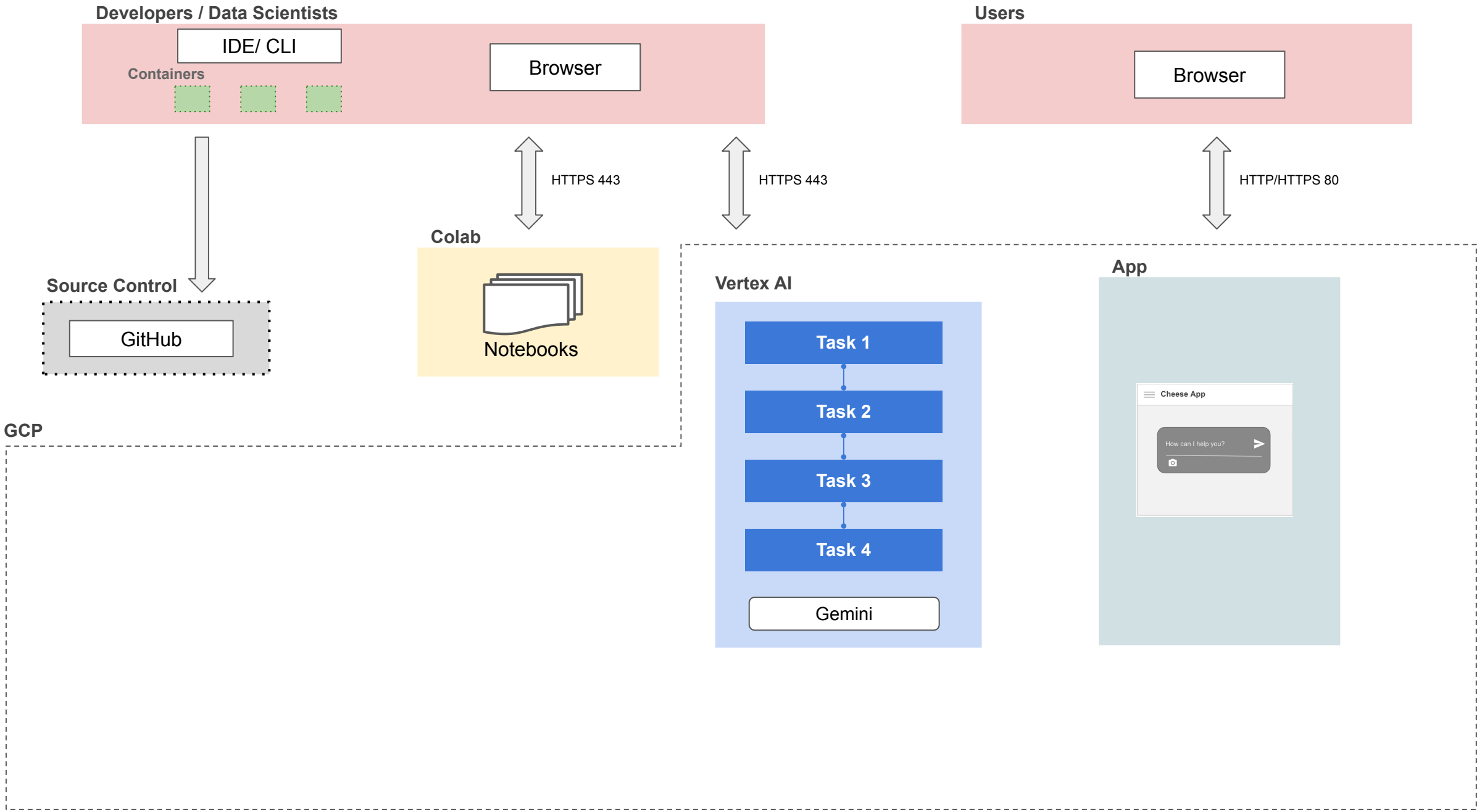
Building a Technical Architecture



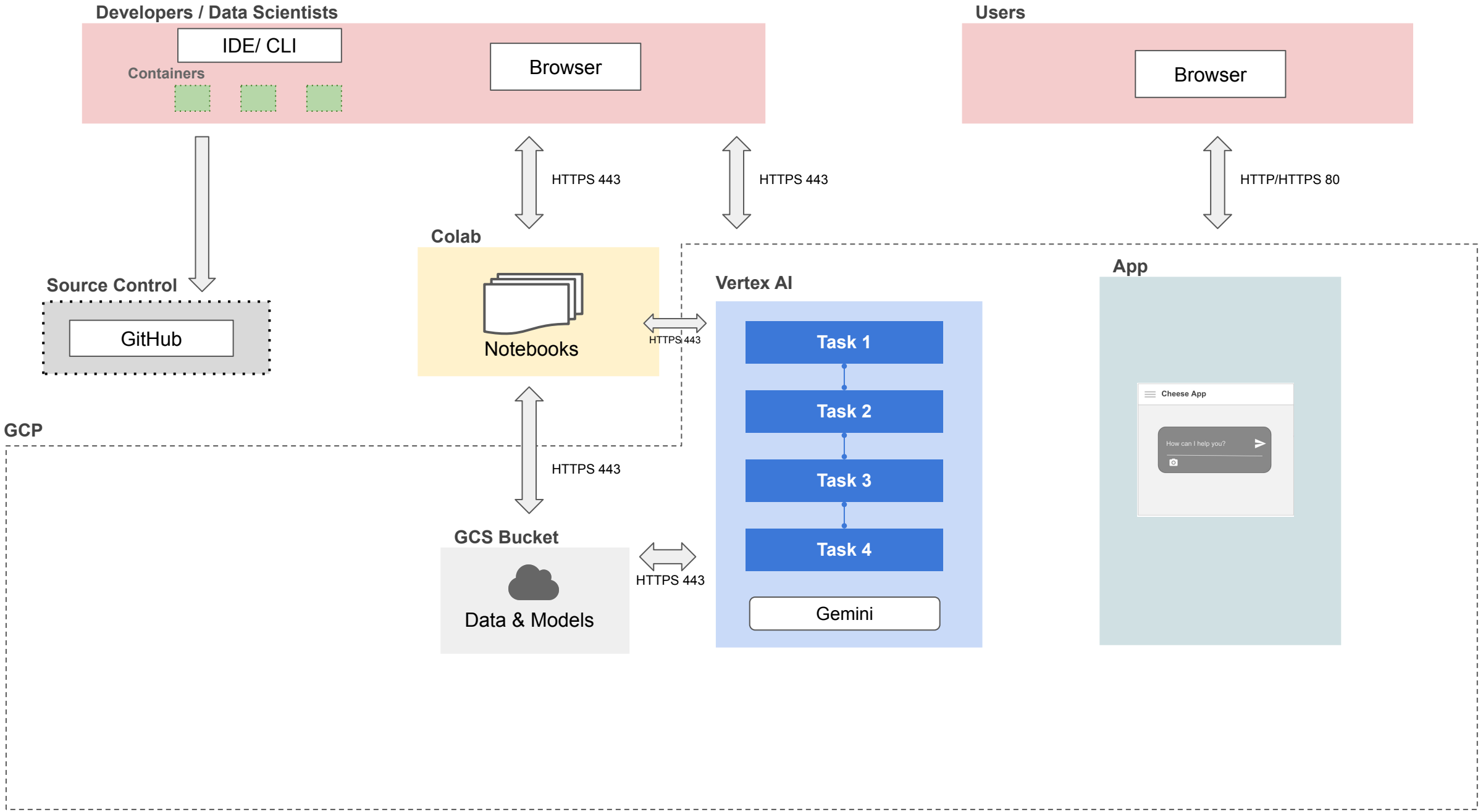
Building a Technical Architecture



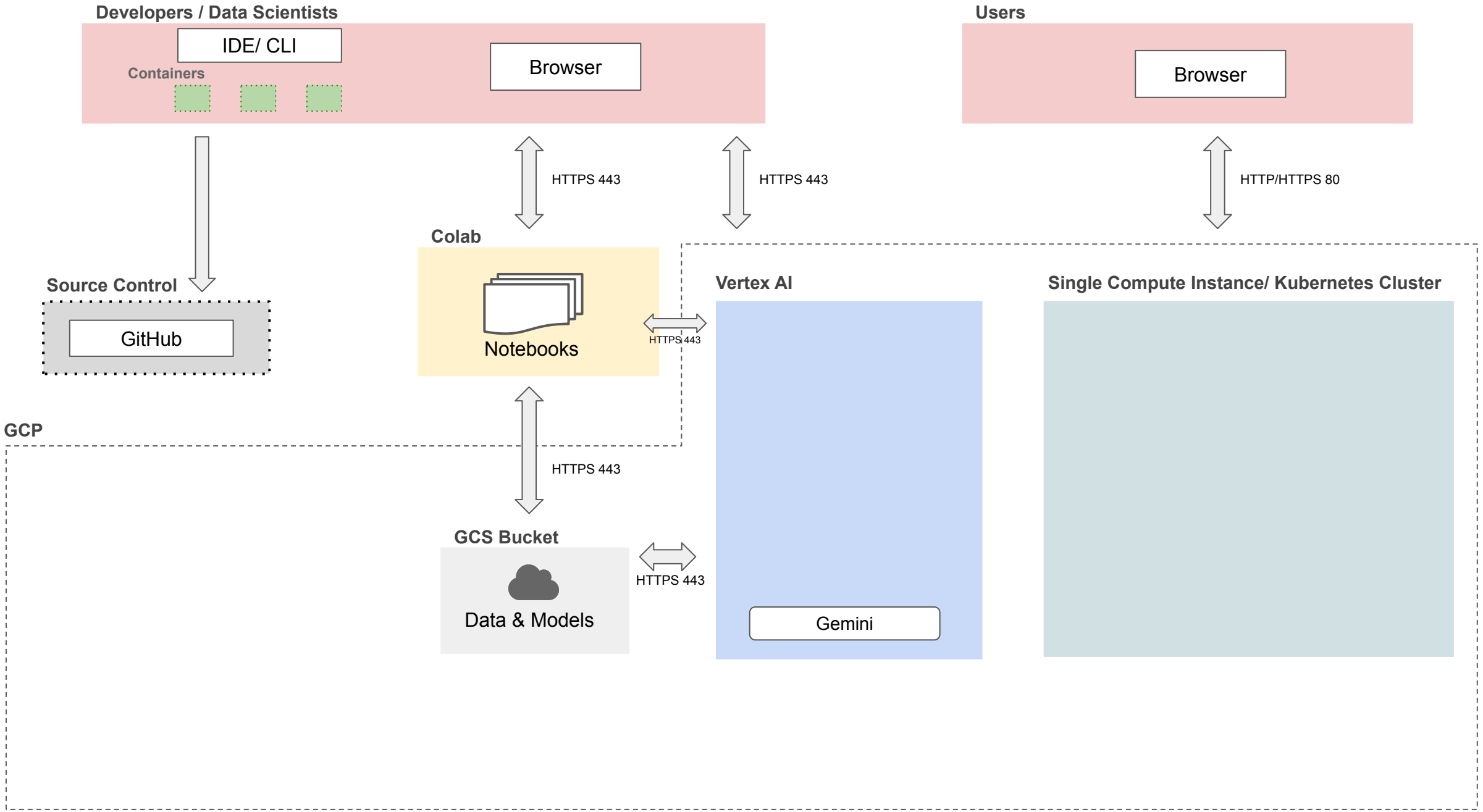
Building a Technical Architecture



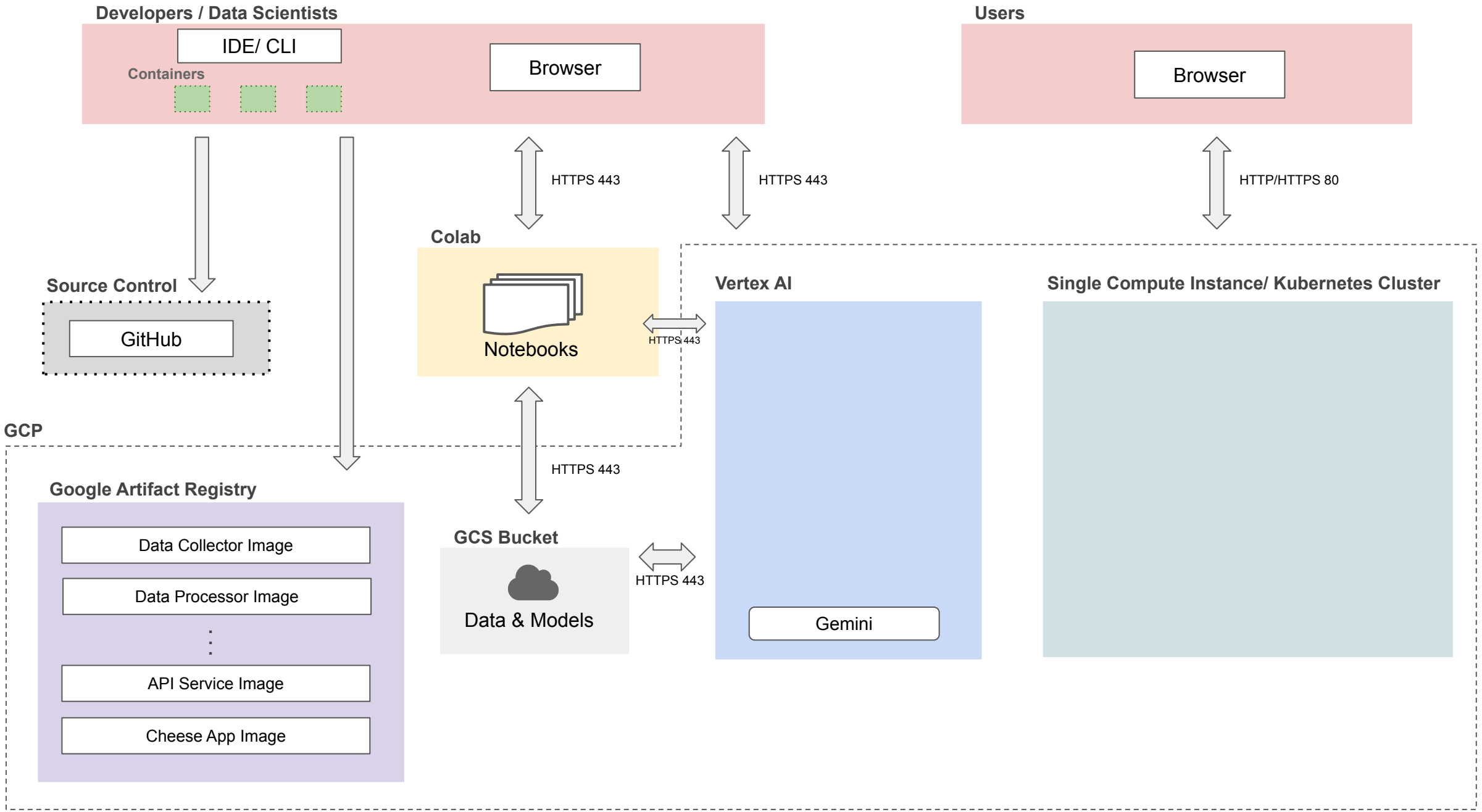
Building a Technical Architecture



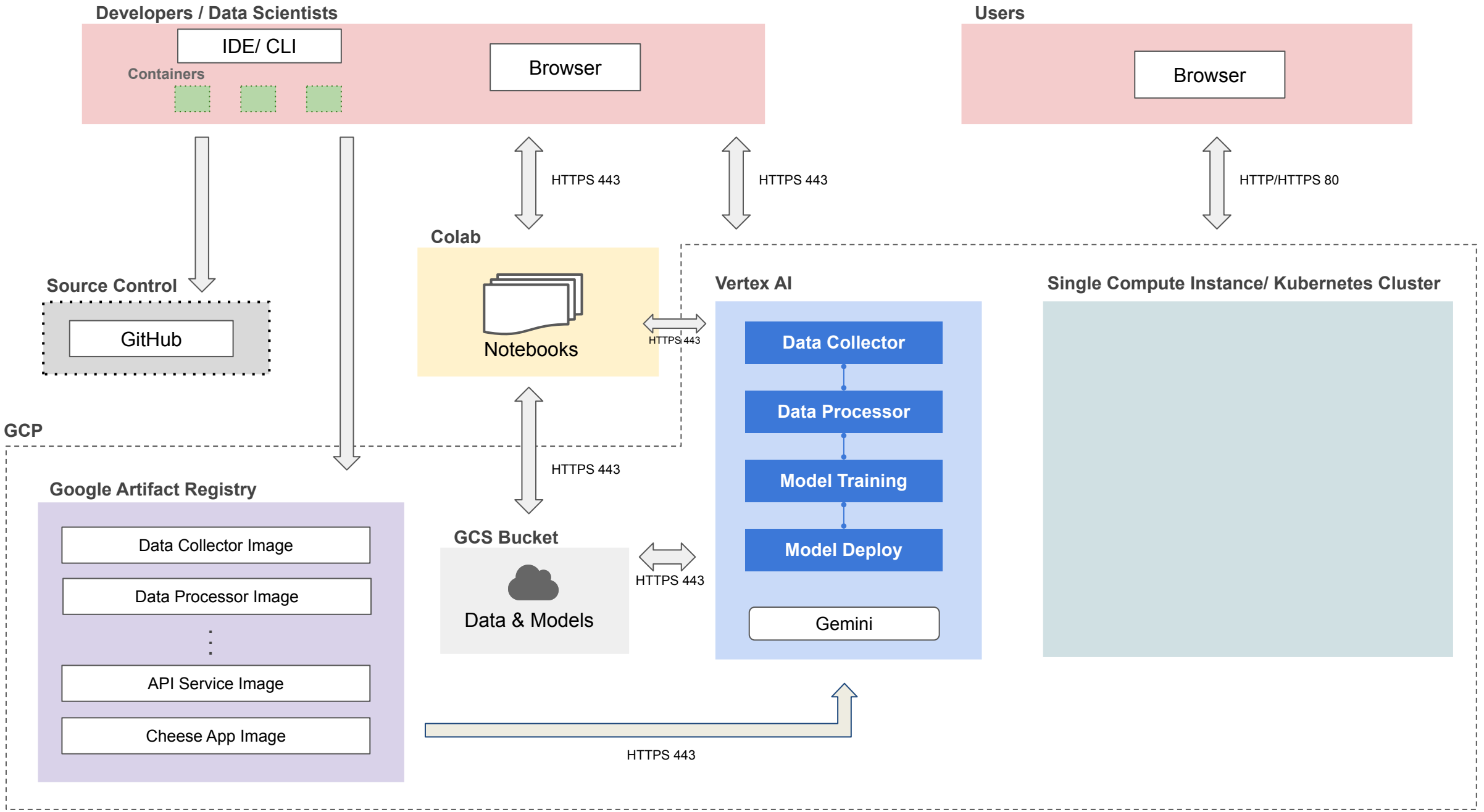
Building a Technical Architecture



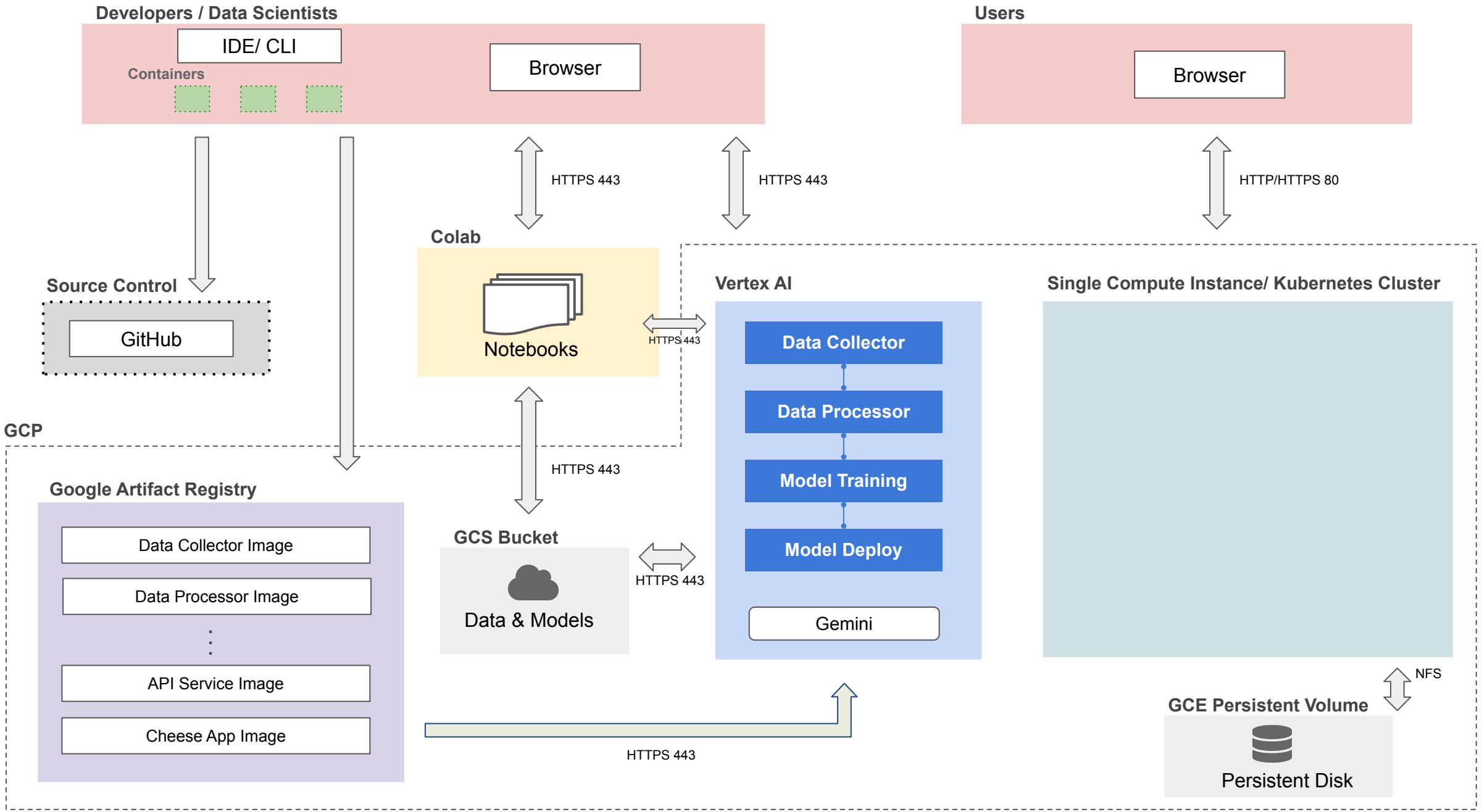
Building a Technical Architecture



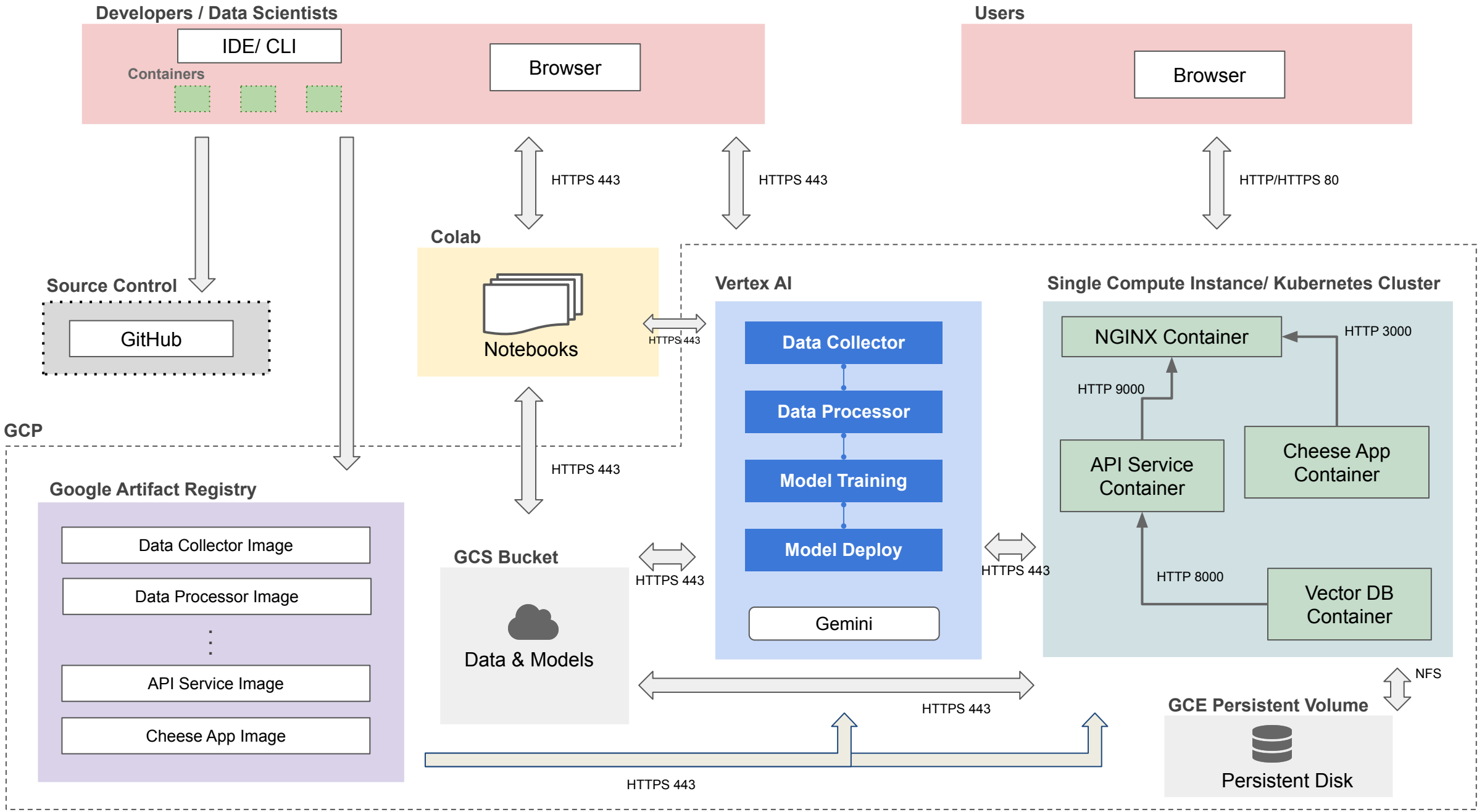
Building a Technical Architecture



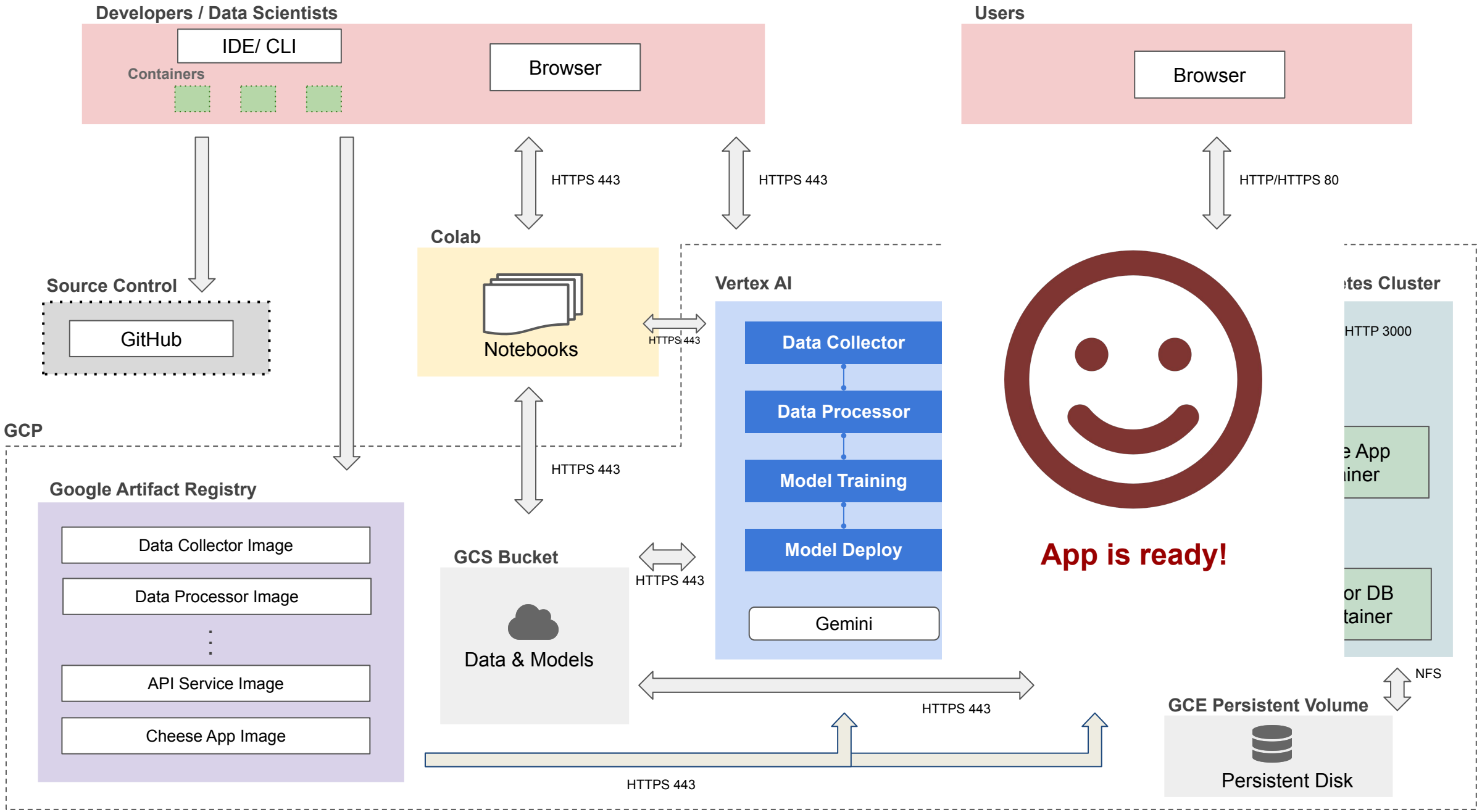
Building a Technical Architecture



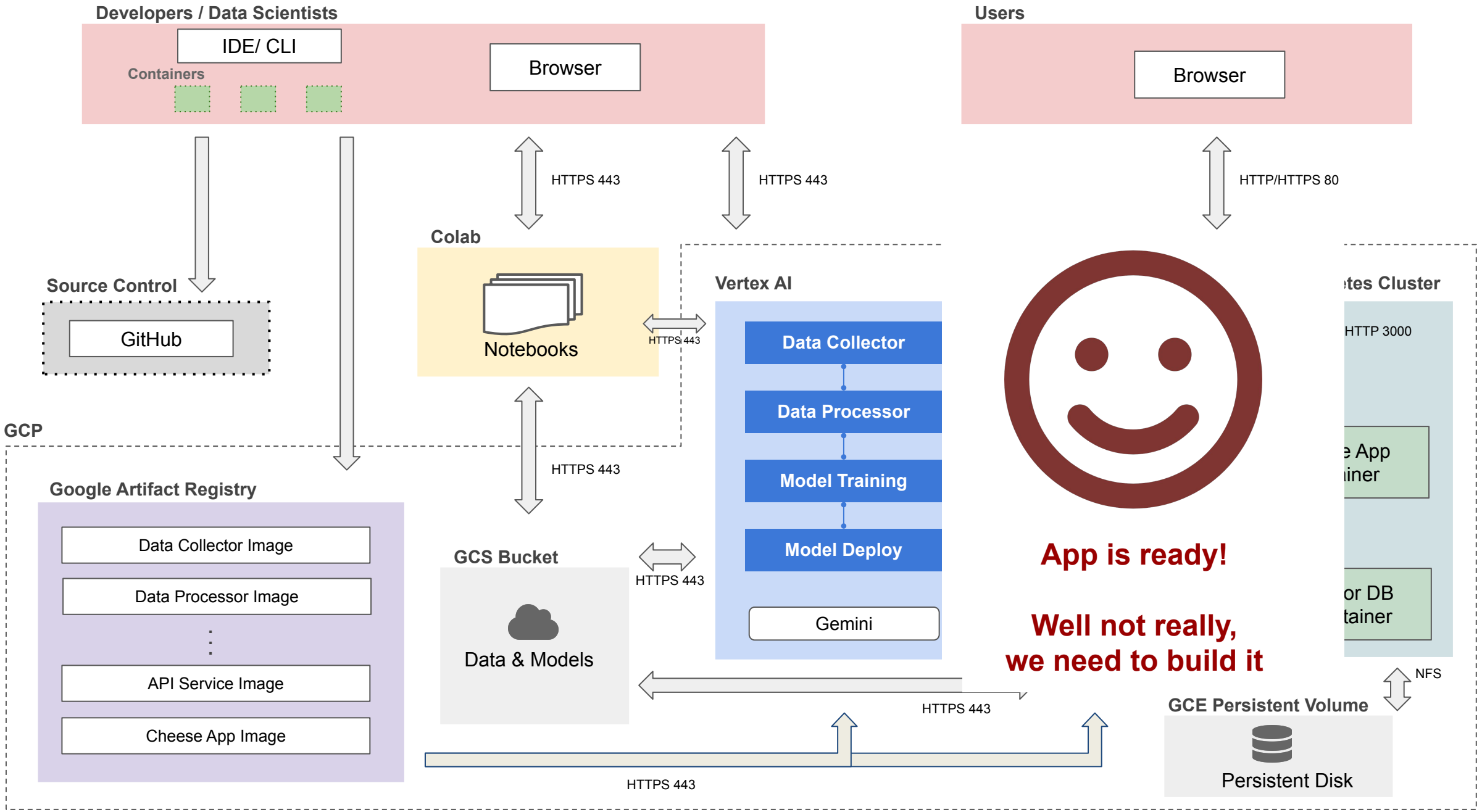
Technical Architecture



Technical Architecture



Technical Architecture



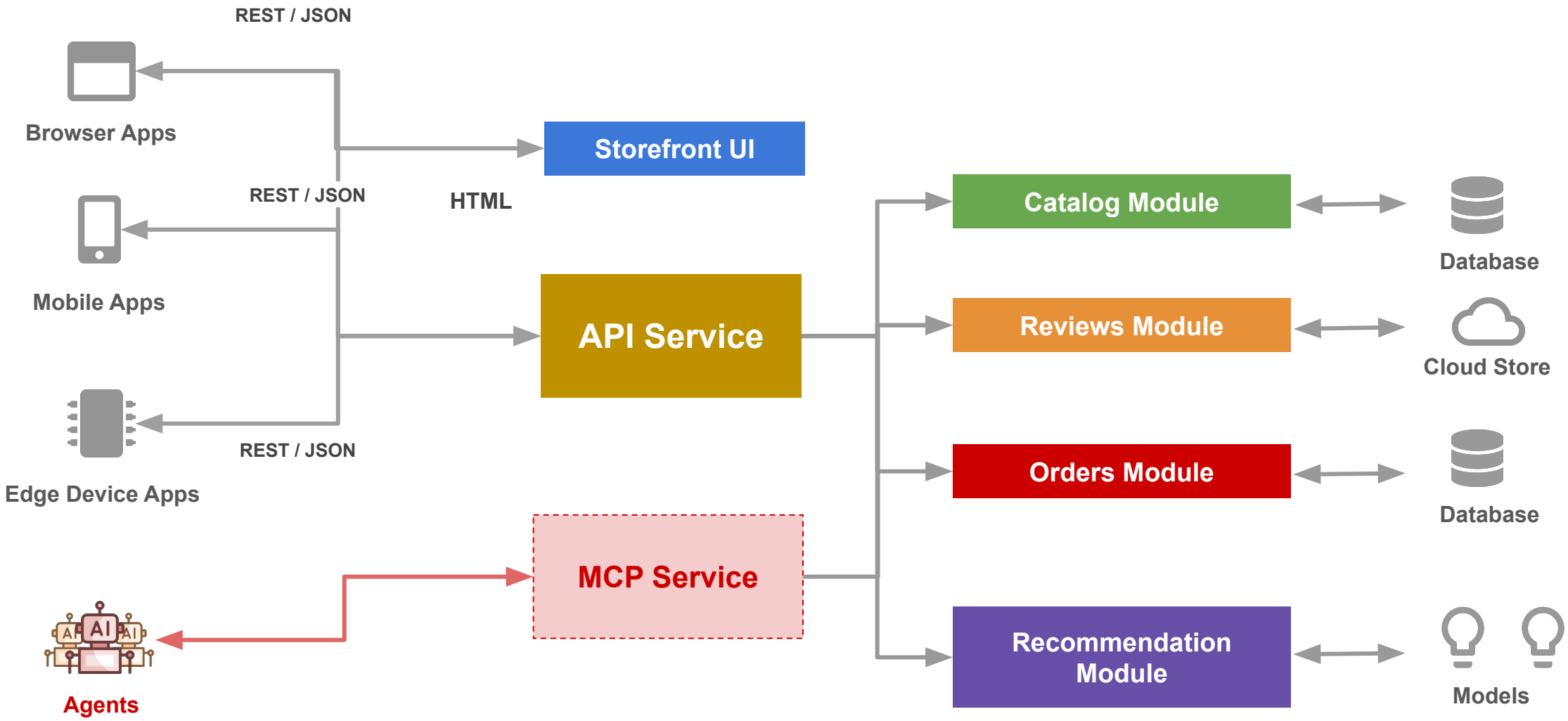
Technical Architecture Building Blocks (Comparison)

	Google Cloud Platform (GCP)	Amazon Web Services (AWS)	Microsoft Azure
Container Registry	Google Artifact Registry	Elastic Container Registry	Azure Container Registry
Object Storage	Google Cloud Storage	S3 Bucket	Azure Blob Storage
AI / ML Platform	Vertex AI	Amazon SageMaker	Azure Machine Learning
LLMs	Vertex AI LLMs	Bedrock LLMs	Azure OpenAI Service
Persistent Storage	GCE Persistent Volume	Elastic Block Store	Azure Disk Storage
Virtual Machines	Compute Instance	EC2 Instance	Azure Virtual Machine
Kubernetes Service	Google Kubernetes Engine	Elastic Kubernetes Service	Azure Kubernetes Service

Outline

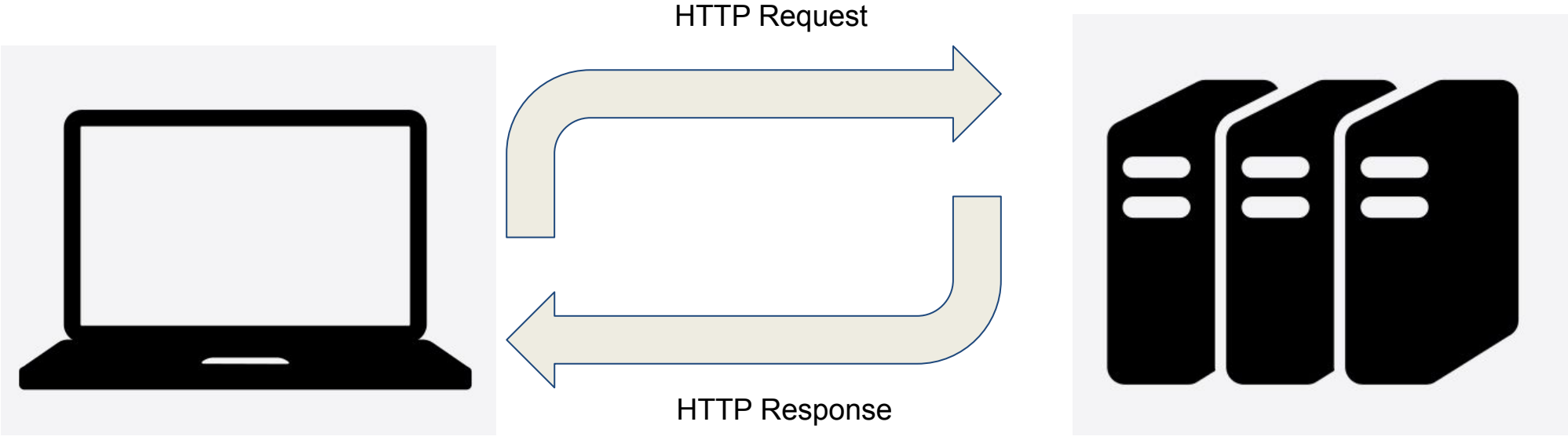
1. Recap
2. Technical Architecture Review
3. **APIs**

Review: Microservice Architecture



Review: What is HTTP?

- HyperText Transfer Protocol: method for **transporting information** where **client** (such as a web browser) makes **request** and web **server** issues a **response**
  - content can be anything from text to images to video
- HTTPS: encryption for **secure** communication over network



Review: What is a port?

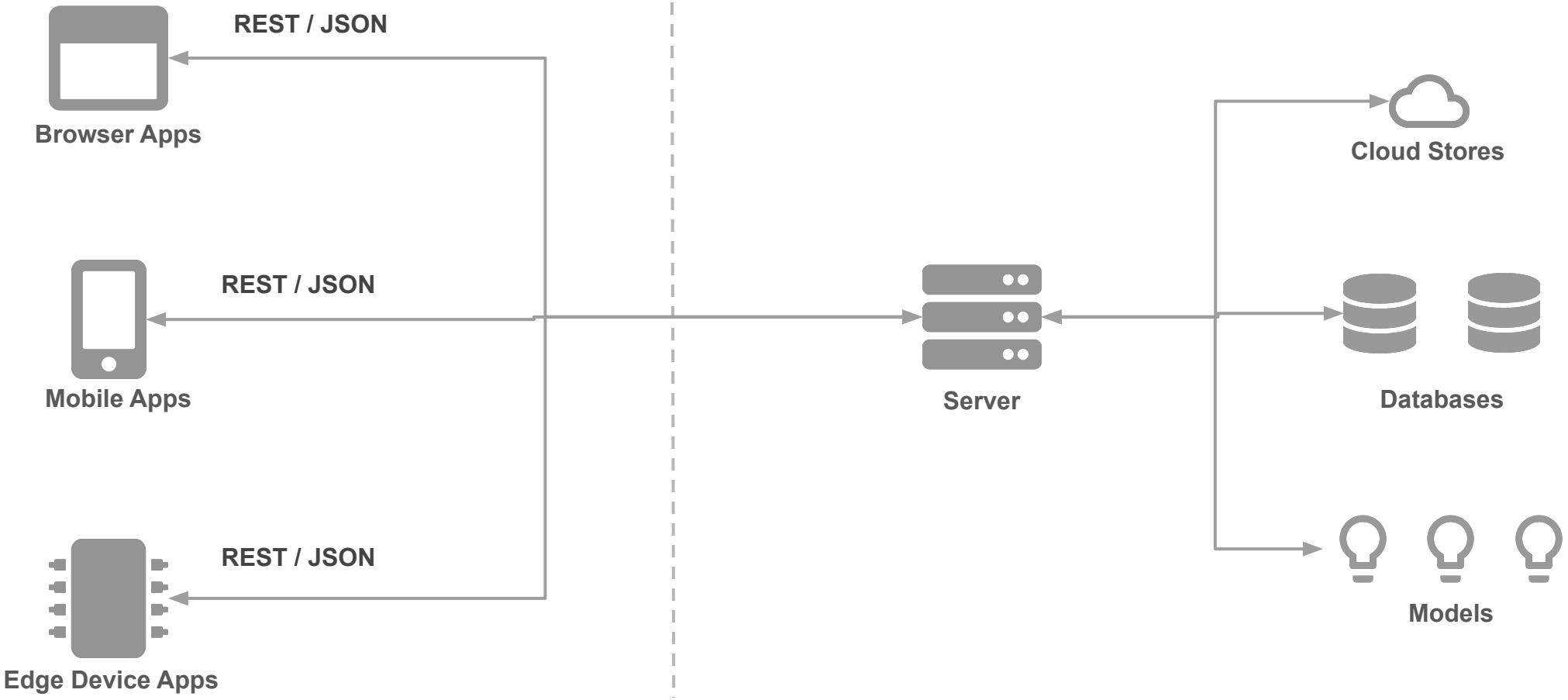
- **communication endpoint** where network connections start and end
- lets **computers differentiate** between different kinds of **data** (emails, webpages, etc.)
  - Port 22 = SSH
  - Port 25 = SMTP (email)
  - Port 80 = HTTP
  - Port 443 = HTTPS

What is an API

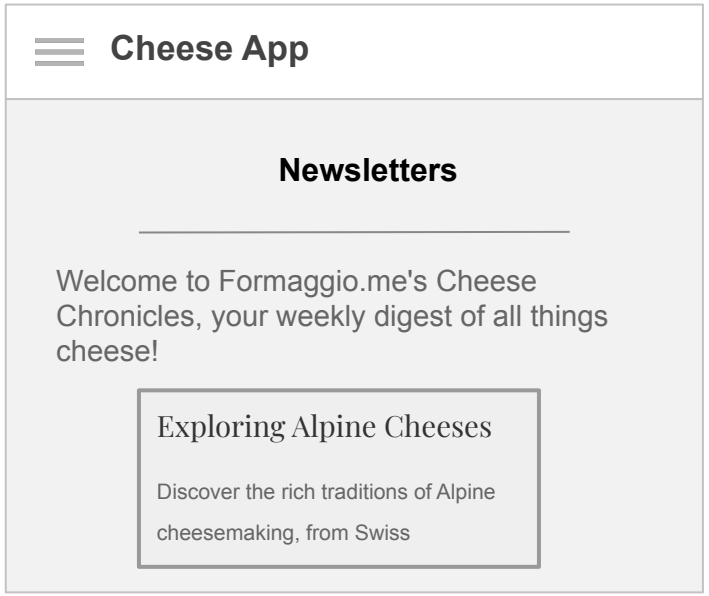
- API is **Application Programming Interface**
- **Web API** is an API that can be access using HTTP/S
- A **REST API** is a Web API that follows the HTTP method constraints - get, post, put, delete
- We will use **FastAPI** a Python framework to build REST APIs

APIs

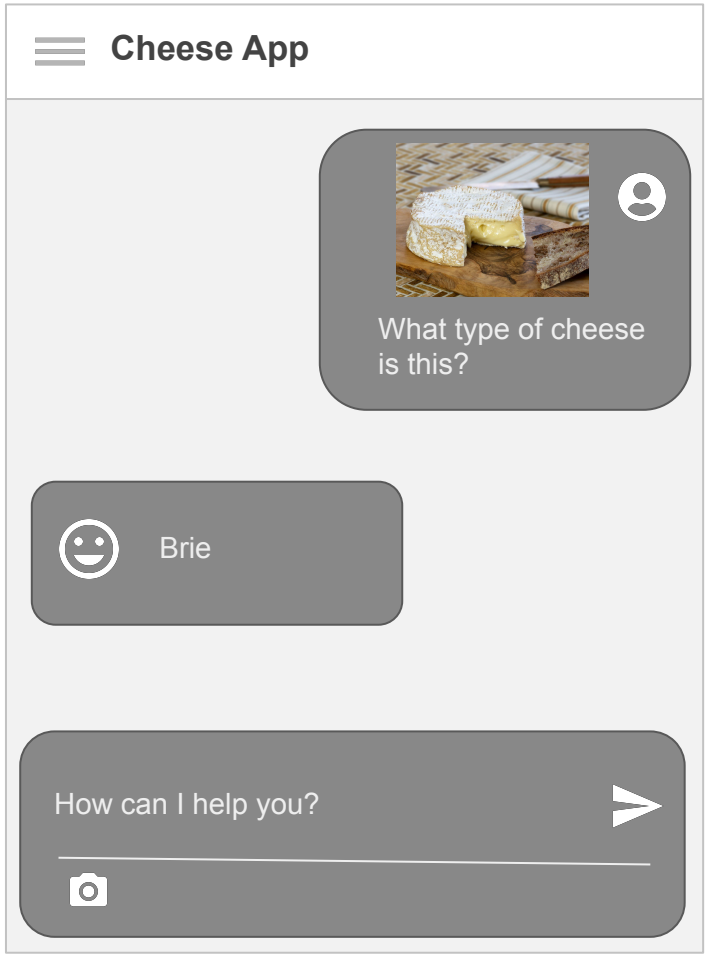
We will be using the term **API** to refer to REST API, which will be used to connect to various components



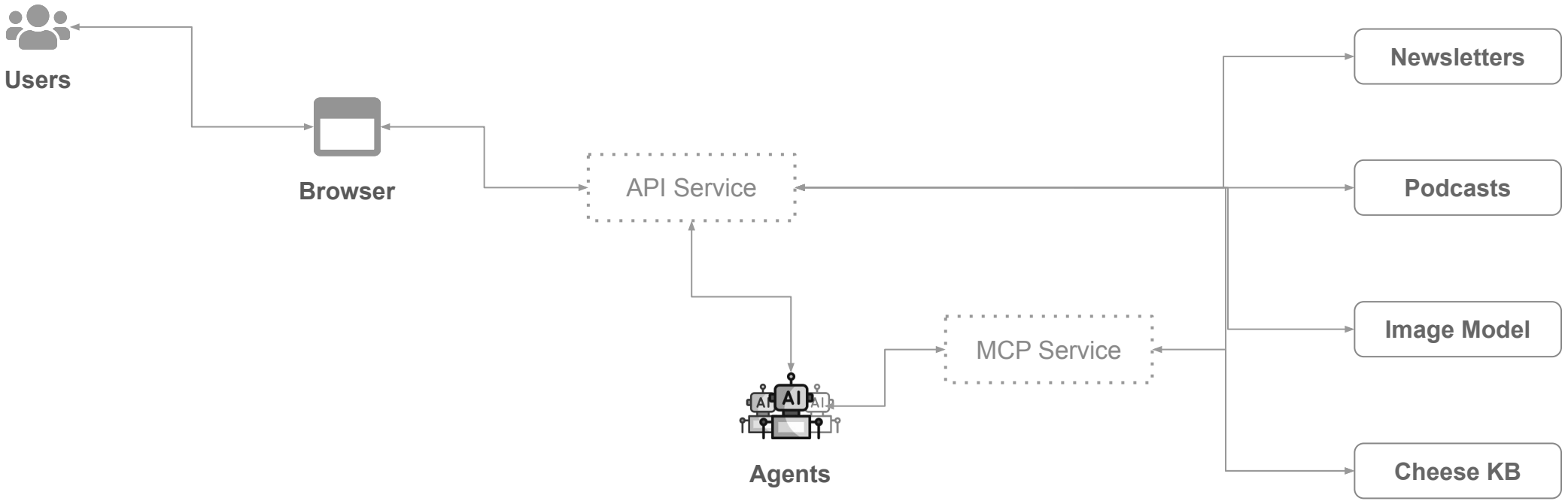
Review: Screenflow & Wireframes



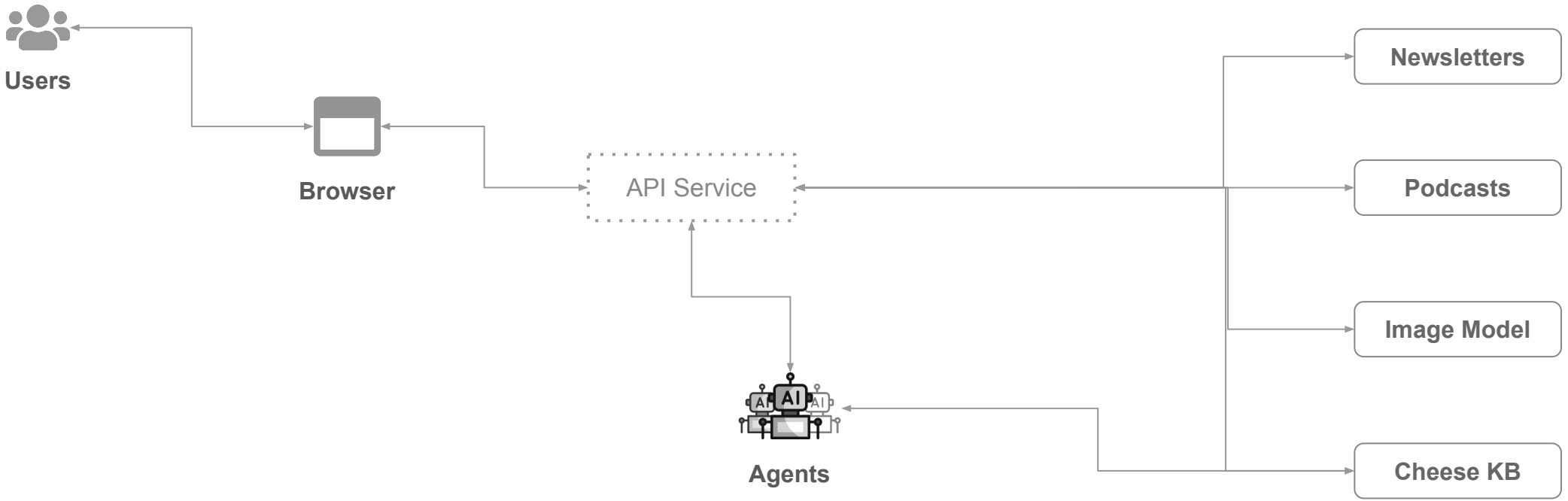




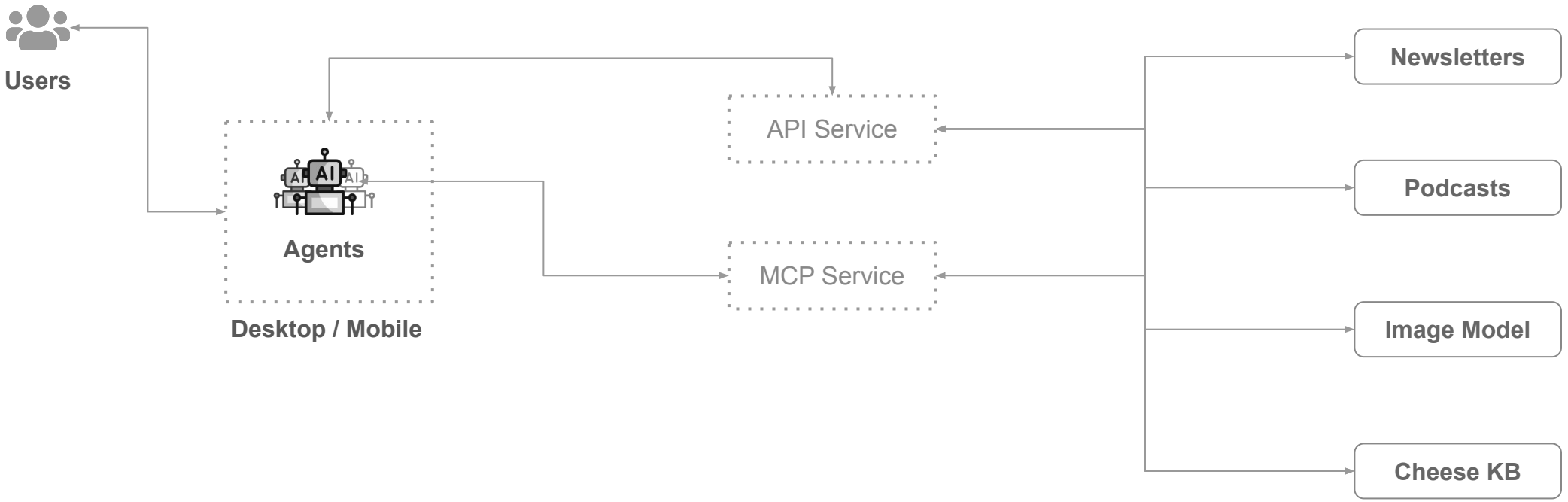
API Design Patterns



API Design Patterns



API Design Patterns

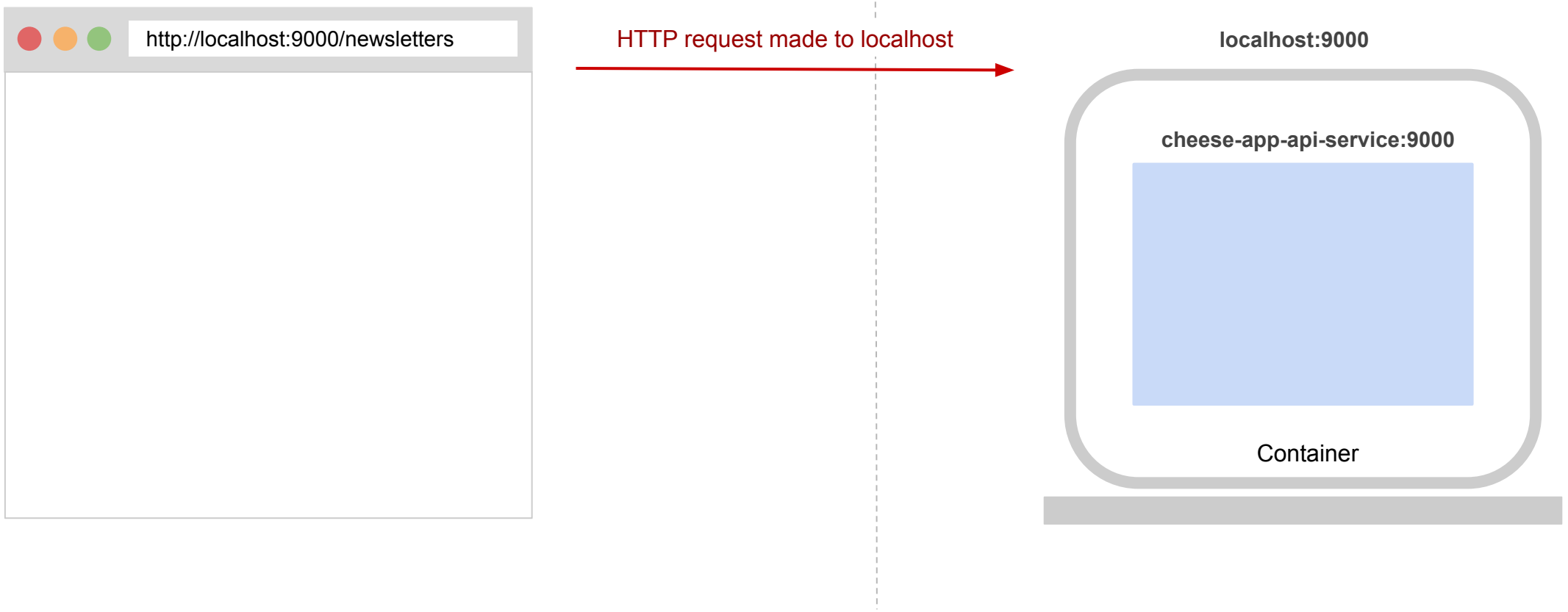


API & MCP

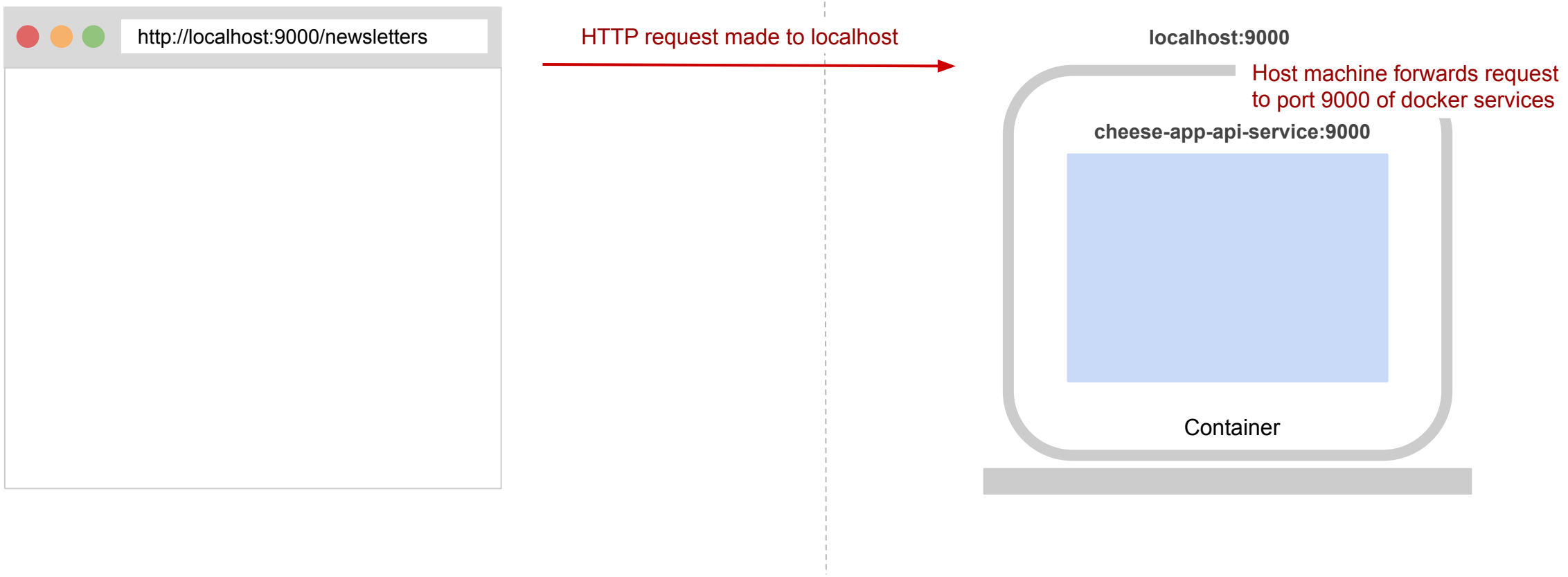
- API is **Application Programming Interface**
- **Web API** can be accessed using HTTP/S
- A **REST API** is a Web API that follows the HTTP method constraints - get, post, put, delete
- **FastAPI** to build REST APIs

- MCP is **Model Context Protocol**
- **Remote MCPs** can be accessed using SSE (Server-Sent Events) over HTTP/S
- MCP provides a **universal standard** for agent-tool communication
- **FastMCP** to build MCPs

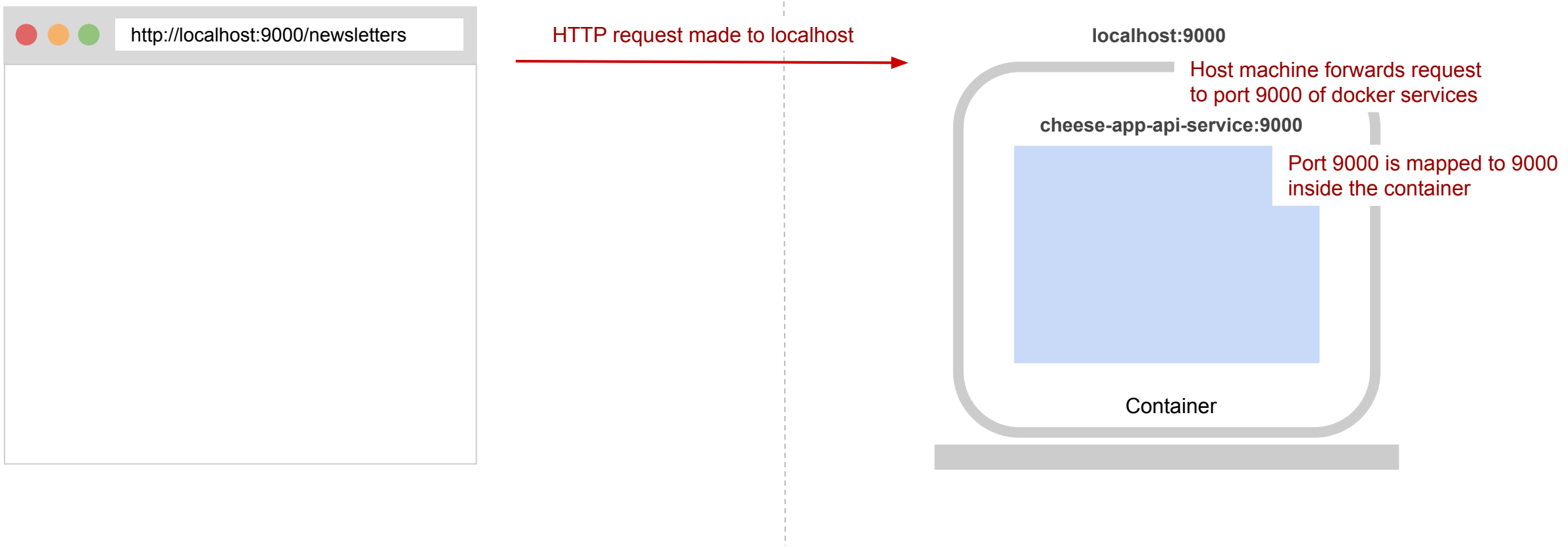
How does an API work



How does an API work



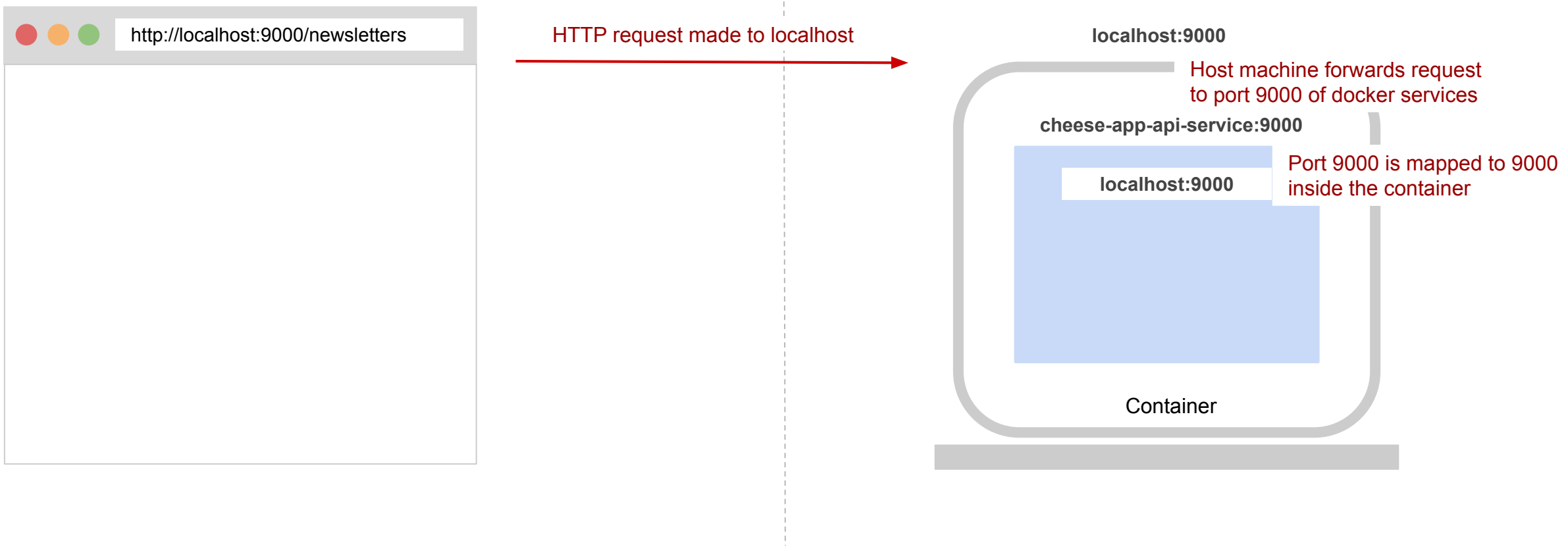
How does an API work



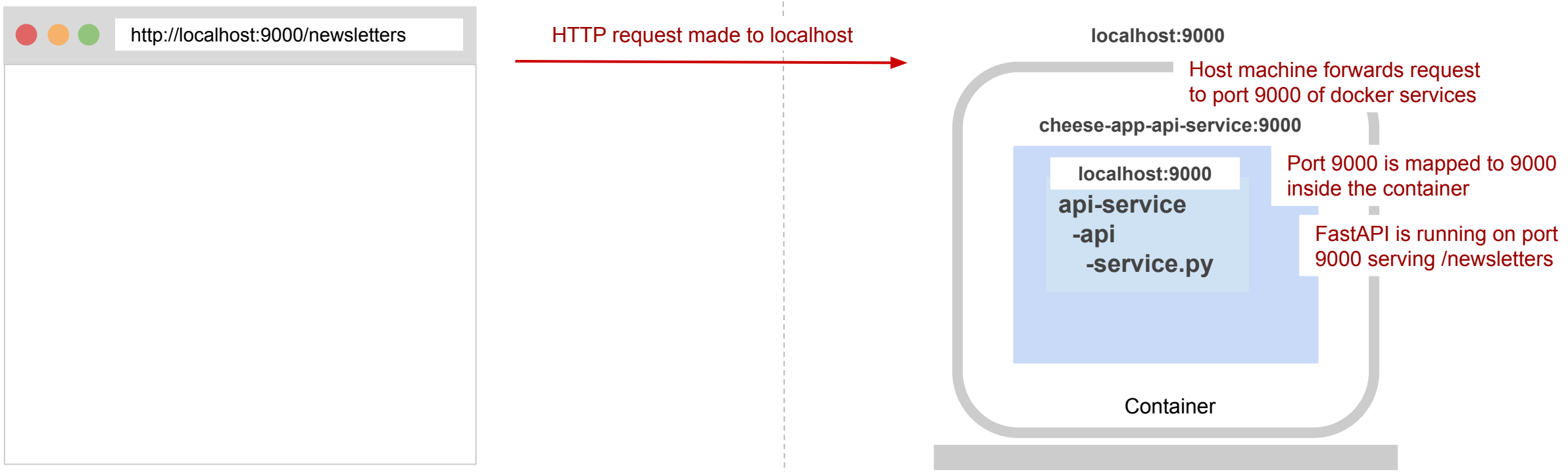
Browser

Local computer / Server

How does an API work



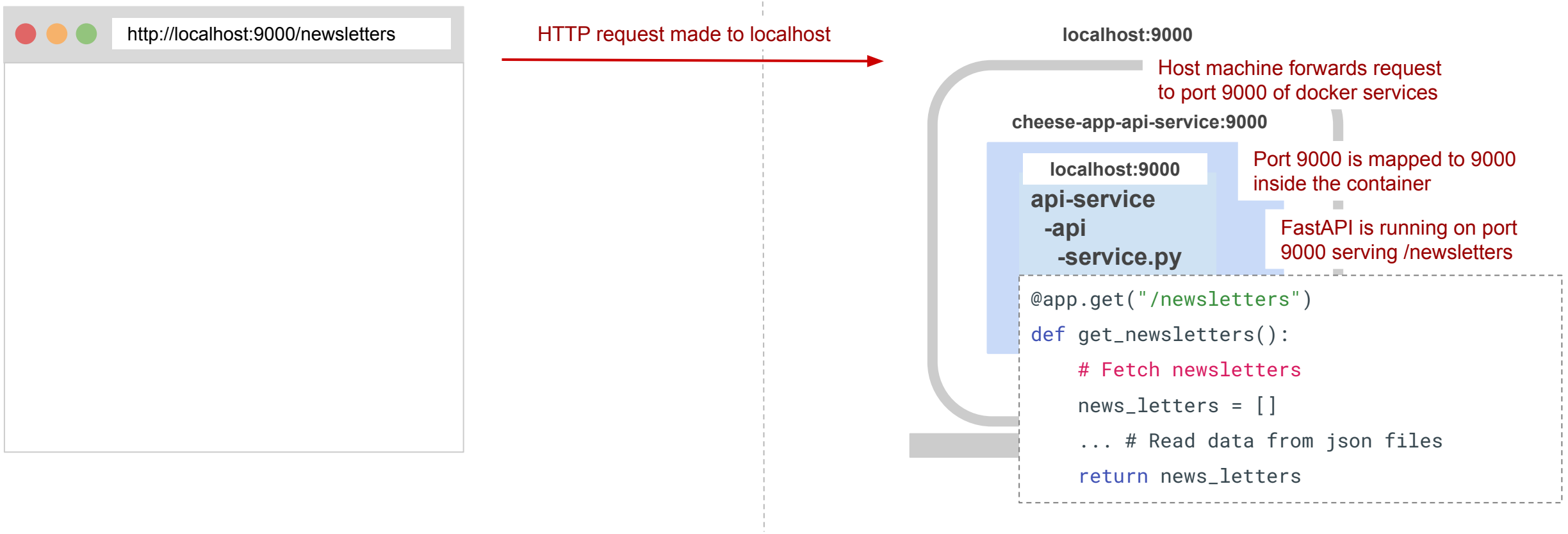
How does an API work



Browser

Local computer / Server

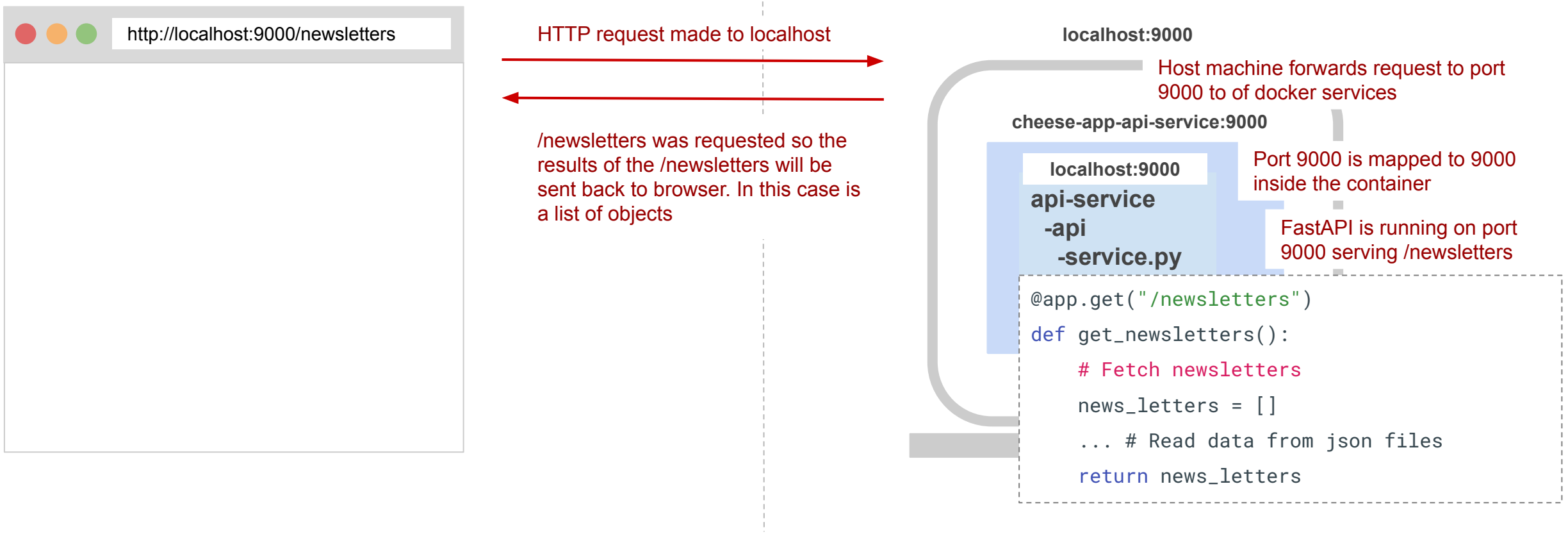
How does an API work



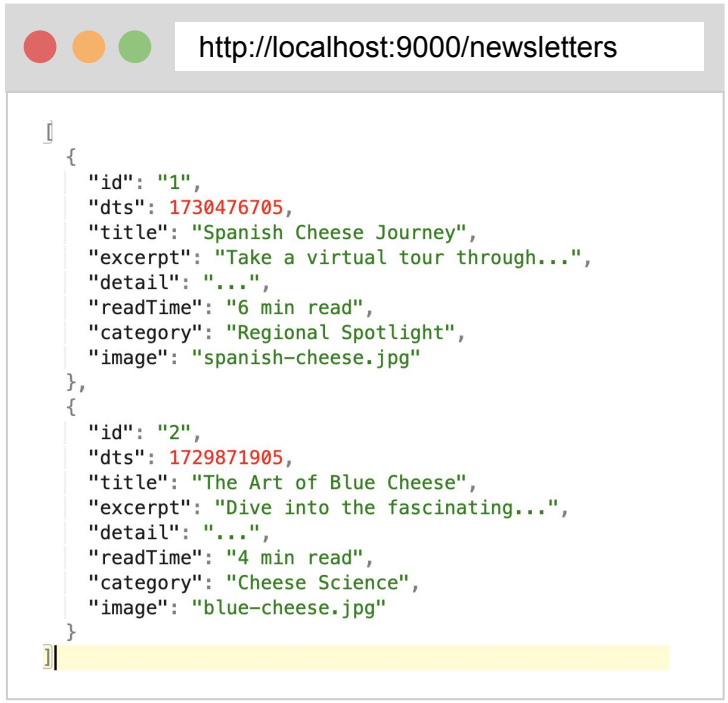
Browser

Local computer / Server

How does an API work

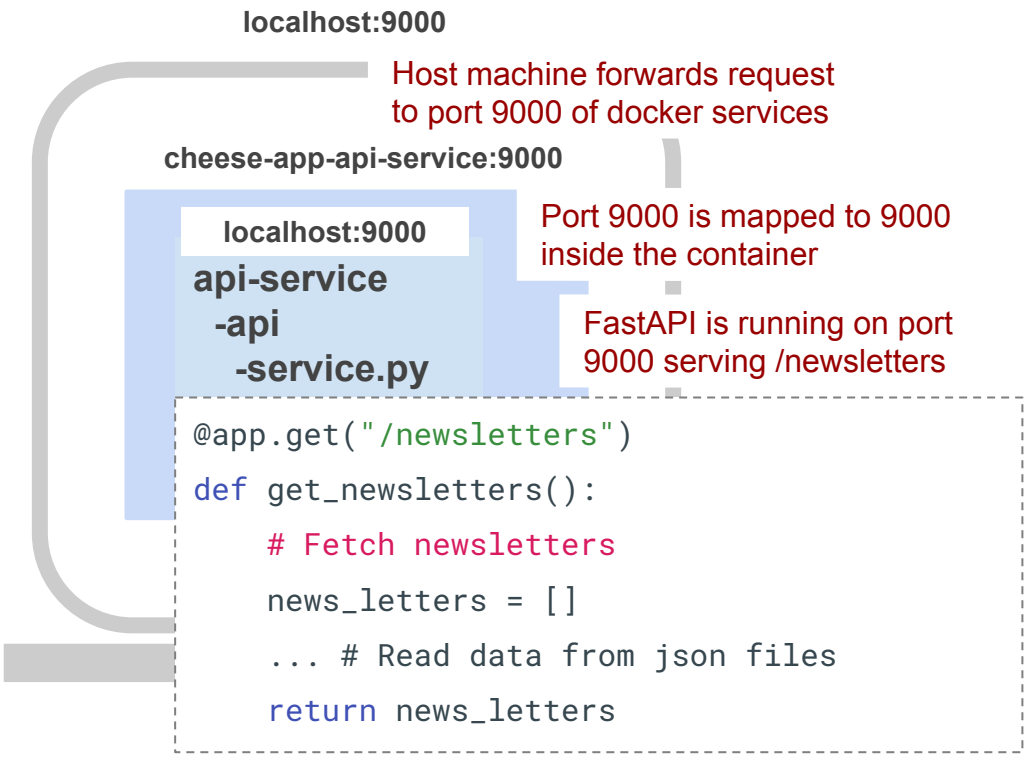


How does an API work



HTTP request made to localhost

/newsletters was requested so the results of the /newsletters will be sent back to browser. In this case is a list of objects



Browser

Local computer / Server

How does an API work (In Production)



HTTPS request made to server

/newsletters was requested so the results of the /newsletters will be sent back to browser. In this case is a list of objects

12.12.12234.34:443

Host machine forwards request to port 80 of docker services

cheese-app-api-service:80

localhost:9000

Port 80 is mapped to 9000 inside the container

api-service
-api
-service.py

FastAPI is running on port 9000 serving /newsletters

```
@app.get("/newsletters")
def get_newsletters():
    # Fetch newsletters
    news_letters = []
    ... # Read data from json files
    return news_letters
```

Browser

GCP Server

Setup & Code Organization

```
/application-root-folder
   /src
        /api-service
        /data-collector
        /data-processor
        /model-training
        /model-deploy
        /ml-workflow
        /frontend-simple
        /frontend-react
        /vector-db
        /postgres-db
/secrets
   - data-service.json
```

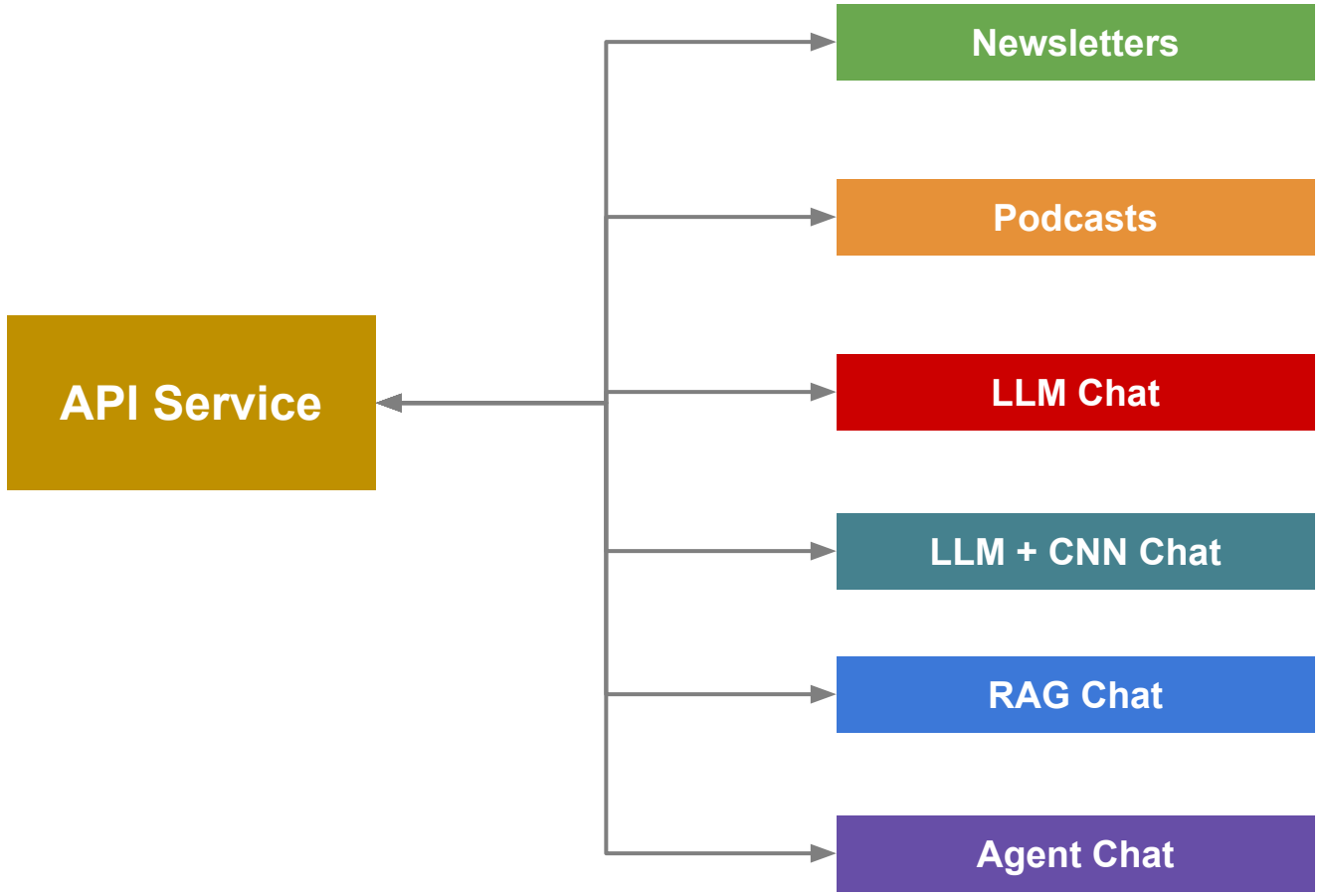


Github root



Secrets (outside git folder)

Tutorial: APIs



Tutorial: APIs

Steps to build Cheese App **APIs**:

- Ensure vector database is running.
- Expose data using an API.
- For detailed instructions, please refer to the following link
  - Cheese App APIs. (https://github.com/dlops-io/cheese-app-v2#setup-environments)

Tutorial: Building Solution Architecture

Steps to build a **Solution Architecture**

- You will work with your project group
- Go to

  https://docs.google.com/presentation/d/1O1j-zcud3vTnD_dZpNPSddYpM8eoMRJ9Cds2wWzjEuU/edit?usp=sharing .
- Duplicate Slides 2, 3, 4 to the end of the slides.
- Put your group name in the slides.
- Identify Process, Execution, State for your project.
- Work on Solution Architecture slide for your project.
- Work on Technical Architecture slide for your project.

THANK YOU